I used ChatGPT-5.1's Thinking mode to work through the coding portions of HW8. The deliberate reasoning made the process slower, but it forced a clear step-by-step plan for each part of the solution. For example, the convolution-based CPU implementation generated a detailed outline during its 1 minute 57 seconds of thinking. In contrast, it was able to one-shot both the GPU implementation and the follow-up GPU questions. Interestingly, after completing the CPU version, it solved the GPU version even faster (about 47 seconds), suggesting it reused the reasoning patterns developed earlier. All the generated code ran correctly and produced the expected graphs.

This is the trace without annotations:
https://chatgpt.com/share/693a7794-8fbc-800f-a446-9f3d3bf84a18

Below is the annotated version trace:

I gave it context for the problem that was given in the notebook along with the todo it needed to solve

This is the context for the code:
SSM Update Rule

We consider an Linear RNN described by the update:

$$
h_{t+1} = W\,h_t + U\,x_t + b
$$

for $t = 0, 1, \ldots, T - 1$. The variables are:

- $h_t \in R^H$, the hidden state at time t.
- $x_t \in R^{N \times D}$, the input at time t.
- $W \in R^{H \times H}$, the recurrent weight matrix.
- $U \in R^{H \times D}$, the input projection matrix.
- $b \in R^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H.

You need to solve the TODOs:

```
def unrolled_ssm_forward(W, U, b, x):
  """
  Unroll the linear RNN in time:
      h_{t+1} = W h_t + U x_t + b
  with initial h_0 = 0.

  Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input sequence over T steps
  Returns:
    h_all: (N, T, H) hidden states for t=1..T
        (h_all[t] corresponds to h_{t+1} in the usual notation).
  """

##############################################################################
  #               TODO: Implement the recurrent pass here                      #

##############################################################################


##############################################################################
  #                       END OF YOUR CODE                                     #

##############################################################################
  return h_all
```

Thought for 14s ›

Here's one clean way to implement the recurrent pass (assuming you're using PyTorch, which matches the rest of your SSM code):

python Copy code

```python
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
             (h_all[:, t] corresponds to h_{t+1} in the usual notation).
    """
    ##############################################################################
    #                   TODO: Implement the recurrent pass here                  #
    ##############################################################################
    import torch

    N, T, D = x.shape
    H = W.shape[0]

    # Allocate tensor for all hidden states
    h_all = torch.zeros(N, T, H, dtype=x.dtype, device=x.device)

    # Initial hidden state h_0 = 0
    h_t = torch.zeros(N, H, dtype=x.dtype, device=x.device)

    # Unroll over time
    for t in range(T):
        x_t = x[:, t, :]               # (N, D)
        # h_{t+1} = h_t W^T + x_t U^T + b
        h_t = h_t @ W.T + x_t @ U.T + b  # (N, H)
        h_all[:, t, :] = h_t
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return h_all
```

It thought for 14s and correctly got the answer in one shot

Key shape checks:

- `h_t` : (N, H)
- `h_t @ W.T` : (N, H) · (H, H)ᵀ → (N, H)
- `x_t @ U.T` : (N, D) · (H, D)ᵀ → (N, H)
- `b` : (H,) broadcasts to (N, H)

And `h_all[:, t, :]` stores `h_{t+1}` for each time step.

This is the context:
Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions
- `make_conv_kernel(W, T)`: This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K. Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)`: This function will take weights W, U, b and input x and return the hidden states h across different time steps.

```
def make_conv_kernel(W, T):
  """
  Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
  the ssm forward pass using conv1d.

  Args:
    W: (H, H) weight matrix
    T: scalar

  Returns:
    kernel_for_conv: (H, H, T) tensor
  """

  ##########################################################################
  #                    TODO: Implement the kernel here                     #

  ##########################################################################


  ##########################################################################
  #                        END OF YOUR CODE                                #

  ##########################################################################
  return kernel_for_conv

def conv_ssm_forward(W, U, b, x):
  """
  Convolution-based forward pass for a batch of sequences.

  RNN update:  h_{t+1} = W h_t + U x_t + b

  Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input (batch=N, time steps=T, input dim=D)

  Returns:
    h_all: (N, T, H) hidden states
  """
  N, T, D = x.shape
  H = W.shape[0]

  s = x @ U.T + b

  s = s.permute(0, 2, 1)

  # Build the kernel with shape (H, H, T).
  kernel = make_conv_kernel(W, T)

  ##########################################################################
  #                    TODO: Implement the convolution here                #

  ##########################################################################

  ##########################################################################
  #                        END OF YOUR CODE                                #

  ##########################################################################
  return h_all
```

In Thinking mode it worked step by step. It restated the goal, built the kernel using the power of two structure, and checked how conv1d actually behaves. It verified shapes and broadcasting, caught that the padding could not give an output length equal to T with a kernel of size T, and fixed this by using a longer convolution and slicing. Overall it showed a clear pattern of starting from the math, checking how the PyTorch API works, spotting mismatches, and correcting them.

Thought for 1m 57s ›

Nice, this is exactly where that divide-and-conquer / powers-of-W trick comes in.

Here's a complete implementation for both TODOs in PyTorch:

python Copy code

```python
import math
import torch
import torch.nn.functional as F

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar, assumed to be a power of 2

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    ###########################################################################
    #                          TODO: Implement the kernel here                #
    ###########################################################################
    H = W.shape[0]

    # T is a power of 2, so log2(T) is an integer
    depth = math.log2(T)
    assert 2 ** int(depth) == T, "T must be a power of 2"
    depth = int(depth)

    # Start with W^0 = I, shape (1, H, H)
    kernel_for_conv = torch.eye(H, dtype=W.dtype, device=W.device)[None, :, :]

    # W_p will hold W^(1), W^(2), W^(4), ...
    W_p = W
    for _ in range(depth):
        # Left block: W_p @ [I, W, W^2, ...] = [W_p, W_p W, W_p W^2, ...]
        # Right block: [I, W, W^2, ...]
        kernel_for_conv = torch.concatenate(
            [W_p @ kernel_for_conv, kernel_for_conv], dim=0
        )
        # Square W_p for the next doubling
        W_p = W_p @ W_p

    # kernel_for_conv now has shape (T, H, H) and contains:
    # [W^{T-1}, W^{T-2}, ..., W^1, W^0]
    # We want the final shape (H, H, T) with the time index last.
    kernel_for_conv = kernel_for_conv.permute(1, 2, 0)
    ###########################################################################
    #                              END OF YOUR CODE                           #
    ###########################################################################
    return kernel_for_conv
```

It seems like the thinking mode added more comments than usual to explain the code more thoroughly

```python
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
      h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    # s_t = U x_t + b, shape (N, T, H)
    s = x @ U.T + b
    # conv1d expects (N, C_in, L); here C_in = H, L = T
    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)
    ##############################################################################
    #                          TODO: Implement the convolution here              #
    ##############################################################################
    # conv1d: weight has shape (C_out, C_in, K) = (H, H, T)
    # We use padding = T-1 so that the first T outputs correspond to h_{1..T}.
    h_conv = F.conv1d(s, kernel, padding=T - 1)  # (N, H, 2T-1)

    # Keep only the first T time steps; these are h_{1}, ..., h_{T}
    h_conv = h_conv[:, :, :T]  # (N, H, T)

    # Return as (N, T, H)
    h_all = h_conv.permute(0, 2, 1)
    ##############################################################################
    #                              END OF YOUR CODE                              #
    ##############################################################################
    return h_all
```

Quick intuition:

- `make_conv_kernel` builds a bank of powers of `W` using exponentiation by squaring, arranged so that along the last axis we have `[W^{T-1}, …, W, I]` (time-reversed).
- With `padding = T-1`, `conv1d` implements the causal sum $h_{t+1} = \sum_{k=0}^{t} W^{t-k} s_k$, and we then slice the first `T` outputs and reshape back to `(N, T, H)`.

The GPU Implementation

Now you want to implement this on the gpu.

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
    """

    ##############################################################################
    #                TODO: Implement the recurrent pass here                     #
    ##############################################################################


    ##############################################################################
    #                            END OF YOUR CODE                                #
    ##############################################################################
    return h_all
```

Thought for 13s ›

You don't actually need anything "special" for GPU here—just make sure any new tensors you create live on the same device as the inputs. PyTorch will run it on GPU automatically if `x`, `W`, `U`, `b` are CUDA tensors.

Here's a clean implementation:

python — Copy code

```python
import torch

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[:, t] corresponds to h_{t+1} in the usual notation).
    """
    ##############################################################################
    #                TODO: Implement the recurrent pass here                     #
    ##############################################################################
    N, T, D = x.shape
    H = W.shape[0]

    device = x.device
    dtype = x.dtype

    # Allocate on the same device/dtype as x (CPU or GPU)
    h_all = torch.zeros(N, T, H, device=device, dtype=dtype)
    h_t   = torch.zeros(N, H, device=device, dtype=dtype)  # h_0 = 0

    for t in range(T):
        x_t = x[:, t, :]                # (N, D)
        # h_{t+1} = W h_t + U x_t + b
        # shapes: (N,H)@(H,H)^T + (N,D)@(H,D)^T + (H,) -> (N,H)
        h_t = h_t @ W.T + x_t @ U.T + b
        h_all[:, t, :] = h_t
    ##############################################################################
    #                            END OF YOUR CODE                                #
    ##############################################################################
    return h_all
```

To actually run this on GPU, just move your inputs once before calling:

python — Copy code

```python
device = torch.device("cuda")
W = W.to(device)
U = U.to(device)
b = b.to(device)
x = x.to(device)

h_all = unrolled_ssm_forward(W, U, b, x)
```

As long as `x` (and the weights) are on CUDA, all ops inside will be GPU ops.

Thought for 47s ›

Here's how you can fill in both TODOs; this will automatically run on GPU as long as `W, U, b, x` are CUDA tensors (since everything we create is on their devices).

python Copy code

```python
import math
import torch
import torch.nn.functional as F

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    ################################################################################
    #                          TODO: Implement the kernel here                     #
    ################################################################################
    H = W.shape[0]

    # T is assumed to be a power of 2
    depth = math.log2(T)
    assert 2 ** int(depth) == T, "T must be a power of 2"
    depth = int(depth)

    # Start with W^0 = I, shape (1, H, H)
    kernel_for_conv = torch.eye(H, dtype=W.dtype, device=W.device)[None, :, :]

    # W_p will hold W^(1), W^(2), W^(4), ...
    W_p = W
    for _ in range(depth):
        # Concatenate powers in a divide-and-conquer fashion
        kernel_for_conv = torch.cat(
            [W_p @ kernel_for_conv, kernel_for_conv], dim=0
        )
        W_p = W_p @ W_p

    # kernel_for_conv currently has shape (T, H, H)
    # We want (H, H, T) with time as the last dimension
    kernel_for_conv = kernel_for_conv.permute(1, 2, 0)
    ################################################################################
    #                                END OF YOUR CODE                              #
    ################################################################################
    return kernel_for_conv
```

It was able to one shot the problem in a much faster time than it took to solve the cpu question

```python
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
      h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    # s_t = U x_t + b, shape (N, T, H)
    s = x @ U.T + b
    # conv1d expects (N, C_in, L) = (N, H, T)
    s = s.permute(0, 2, 1)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)
    ##########################################################################
    #                          TODO: Implement the convolution here          #
    ##########################################################################
    # conv1d weight: (C_out, C_in, K) = (H, H, T)
    # Use padding=T-1 to get causal convolution and then take first T steps
    h_conv = F.conv1d(s, kernel, padding=T - 1)   # (N, H, 2T-1)
    h_conv = h_conv[:, :, :T]                     # (N, H, T) → h_{1..T}

    # Return as (N, T, H)
    h_all = h_conv.permute(0, 2, 1)
    ##########################################################################
    #                              END OF YOUR CODE                          #
    ##########################################################################
    return h_all
```

No extra GPU magic needed: if `x`, `W`, `U`, `b` are on `cuda`, then `kernel_for_conv`, `s`, and `h_all` will also live on GPU and everything runs there.